

PBT — Pushback Training Platform Handover

The accounts you need to own, what each one powers, and the step-by-step sequence to take full control of the platform.

What's in this document

- 01 Executive summary — what you are taking ownership of
- 02 Architecture at a glance — which service hosts what
- 03 Accounts to create — Supabase, Netlify, Google AI Studio, GitHub, email provider, Notion
- 04 Environment variables — the complete reference
- 05 Handover sequence — transfer vs. rebuild, step by step
- 06 Verification — how to prove the platform is healthy before go-live
- 07 Security & ownership checklist
- 08 Ongoing operations, costs and monitoring
- 09 Known outstanding work
- 10 Handover sign-off

How to use this document. Section 3 tells you which accounts to create and why. Section 5 is the runbook: follow it top to bottom on handover day. Everything else is reference material you will come back to. The companion admin documentation lives in Notion (see Section 3.6).

Executive summary

PBT (Pushback Training) is a mobile-first web app that trains veterinary teams to handle client pushback. Team members role-play against an AI customer (text or live voice), receive an ACT-based scorecard (Acknowledge · Clarify · Transform, plus Empathy and Rapport), and see their progress over time. A **Fecal Scan** tool scores a stool photo against the Royal Canin Fecal Scoring charts (a supportive aid for technicians, not a diagnostic). An ECHO personality quiz (Activator · Energizer · Analyzer · Harmonizer) personalises the experience. A separate **admin dashboard** at `/admin` lets your team manage users and roles, tune the AI simulation, maintain the scenario library and knowledge base, send branded emails, toggle features without a deploy, and monitor quality and cost.

What ownership means in practice

The platform is made of five hosted services plus the source code. Today they all sit in Rethink Reality accounts. After handover they must sit in *your* accounts so that billing, data, credentials and uptime are fully under your control. No account is optional except Notion (documentation only) and, strictly speaking, the email provider (without it, admin invitations and password resets cannot be delivered).

Supabase
Database, user accounts, admin roles, telemetry, knowledge base with vector search. All persistent data lives here.

Netlify
Hosts the web app and the admin dashboard, runs the 38 server-side functions (including every AI call), holds every secret as an environment variable.

Google AI Studio (Gemini API)
Every AI feature: customer role-play, scoring, coach hints, Pet Vision photo analysis, Fecal Scan, live voice, and knowledge-base embeddings. The key is used only on the server.

GitHub
The source code. Netlify deploys automatically from the `main` branch.

Email provider (Resend or SMTP)
Delivers admin invitations, role-change notices, welcome and password-reset emails through your own domain.

Notion
Hosts the admin documentation system that accompanies this handover. Not part of the running platform.

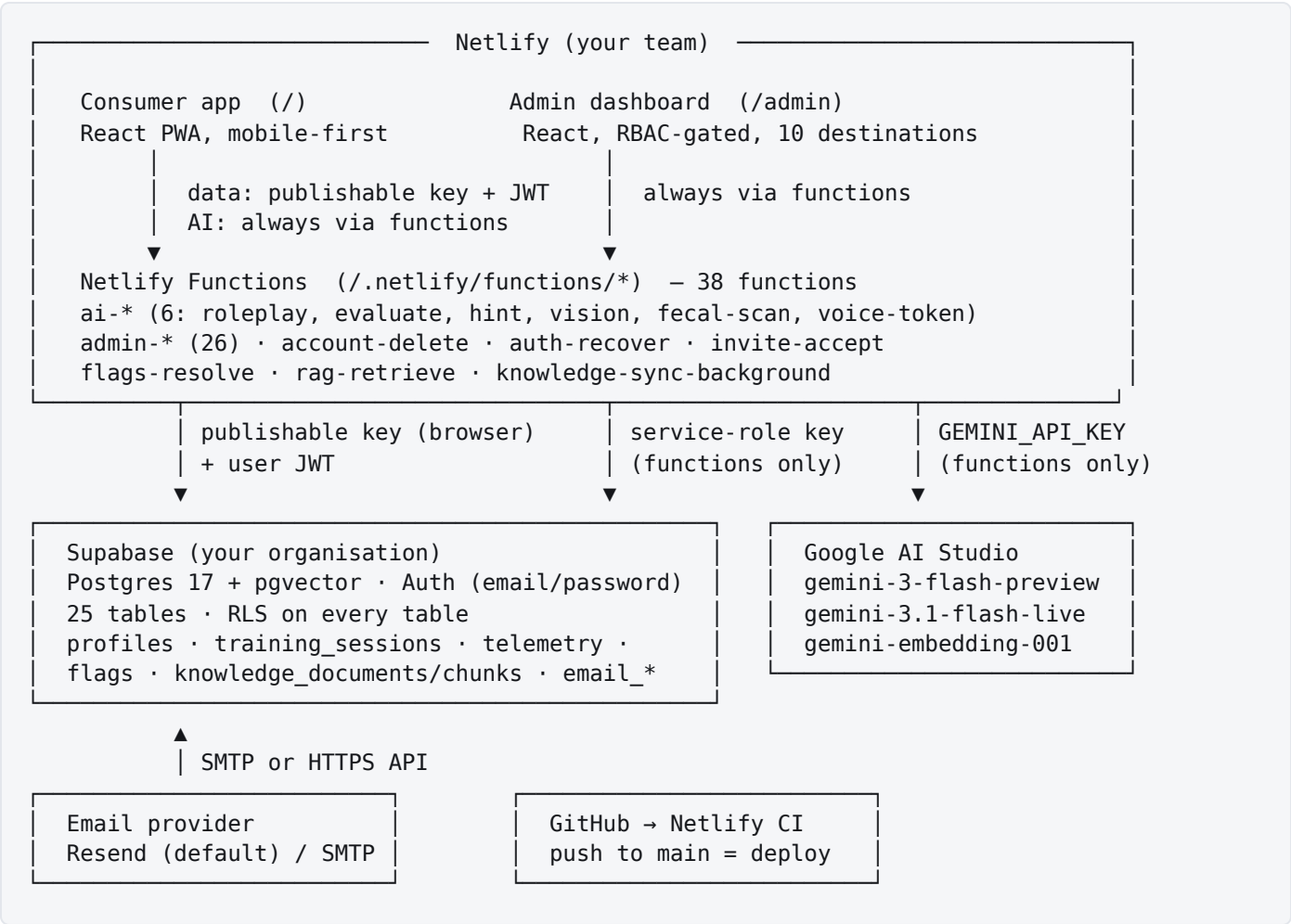
Current identifiers (before handover)

ITEM	VALUE TODAY	AFTER HANDOVER
------	-------------	----------------

Production URL	https://rc-pbt.netlify.app (admin at /admin)	Your Netlify site and, optionally, your custom domain
Netlify site	rc-pbt on the Rethink Reality team (Pro plan)	Transferred to your Netlify team
Supabase project	pbt-pushback-training (ref pkouwgnfwerlllqghjjs, us-east-1, Postgres 17)	Transferred to your Supabase organisation
Source repository	github.com/RethinkRealityAI/PBT, deploy branch main	Transferred to your GitHub organisation
Gemini API key	Rethink Reality Google Cloud project	Replaced with a key from your Google AI Studio account
Admin owners	3 owner accounts (Rethink Reality)	Your owner accounts; ours removed at close-out

Architecture at a glance

Every request starts in the browser on Netlify. Data and authentication go to Supabase. Every AI call is made by a Netlify Function, so the Gemini key never reaches a browser; live voice connects from the device with a single-use token that a Function mints for each session.



Who holds which secret

CREDENTIAL	WHERE IT LIVES	EXPOSURE
Supabase publishable (anon) key	Netlify env → built into the browser bundle	Public by design; Row Level Security protects data
Supabase service-role key	Netlify env → Netlify Functions only	SECRET Bypasses RLS. Never in the browser.

Gemini API key	Netlify env → Netlify Functions only (read at runtime, never built into the bundle)	SECRET Never in the browser. Voice mode gets a single-use, short-lived token per session instead. A build check fails if a key-shaped string ever appears in the bundle.
Email credentials (Resend key / SMTP password)	Entered in Admin → Email → Settings, encrypted at rest in Supabase (or set as env vars)	SECRET Never returned to the browser.
EMAIL_SECRET_KEY	Netlify env → functions only	SECRET Encrypts the email credentials above.

Accounts to create

Create every account with a **shared, role-based company identity** (for example `platform@yourclinic.com`) rather than an individual's personal login, enable two-factor authentication on each, and store the credentials in your password manager. Then add named team members as additional users. This prevents lock-out when a person leaves.

#	SERVICE	REQUIRED?	WHAT IT HOSTS / PROVIDES IN PBT	SIGN UP AT
3.1	Supabase	REQUIRED	Database, authentication, admin roles, all user and telemetry data, knowledge base	supabase.com
3.2	Netlify	REQUIRED	Web hosting for app + admin, serverless functions, environment variables, deploys, custom domain	netlify.com
3.3	Google AI Studio	REQUIRED	Gemini API key that powers every AI feature and knowledge-base embeddings	aistudio.google.com
3.4	GitHub	REQUIRED	Source code; Netlify builds from it	github.com
3.5	Resend (or any SMTP provider)	STRONGLY ADVISED	Delivery of invitation, role-change, welcome and password-reset emails	resend.com
3.6	Notion	OPTIONAL	Admin documentation system (not part of the running platform)	notion.com

3.1 Supabase — database, authentication and data

What it provides. Supabase hosts the Postgres database and the authentication service. Every piece of persistent data in PBT lives here: user profiles and ECHO results, completed training sessions and scores, admin roles and invitations, feature flags, the scenario library, the ingested knowledge base with vector embeddings (pgvector), AI telemetry, navigation analytics, feedback, platform reports, audit log, and email settings and delivery log. Supabase Auth handles email/password sign-in for both trainees and admins.

Plan guidance. The Free tier pauses inactive projects after a week and has no backups; use at least the **Pro** plan for production so you get daily backups, no pausing, and support. Check current pricing at supabase.com/pricing.

Steps

- 1 Create an organisation for your company at supabase.com (Dashboard → New organisation). Enable 2FA on the account.

- 2 **Preferred:** accept the *project transfer* from Rethink Reality (Section 5, Path A). The existing project moves into your organisation with all data intact. **Alternative:** create a new project named `pbt-pushback-training` in region `us-east-1` (Path B), then run the 23 migration files in filename order.
- 3 Confirm the `vector` extension is enabled (Database → Extensions). The knowledge base and retrieval depend on it.
- 4 Authentication → Providers: *Email* enabled; *Confirm email* switched **off** (the product signs users in immediately; the in-app verification flow stays behind a code flag).
- 5 Authentication → URL configuration: set *Site URL* to your production URL and add these redirect URLs: `https://<your-domain>/reset-password`, `https://<your-domain>/admin/reset`, `https://<your-domain>/admin/invite`.
- 6 Collect from Project Settings → API: the *Project URL*, the *publishable (anon)* key, and the *service-role* key. These become the Netlify environment variables in Section 4.

The service-role key bypasses every security rule. It belongs only in Netlify environment variables (functions scope). If it is ever pasted into a chat, email or ticket, rotate it immediately in Project Settings → API.

3.2 Netlify — hosting, functions and secrets

What it provides. Netlify builds the app from GitHub on every push to `main`, serves the consumer app at `/` and the admin dashboard at `/admin`, runs the 38 Netlify Functions (every AI call, all admin data access, account deletion, invitations, password recovery, flag resolution, knowledge retrieval and automatic knowledge-base seeding), stores every environment variable and secret, and provides HTTPS and custom domains.

Plan guidance. The site runs today on a Pro team. The Free tier works technically, but Pro adds team seats, more function invocations and bandwidth, and password-protected deploy previews. Check current limits at netlify.com/pricing.

Steps

- 1 Create a Netlify team for your company and enable 2FA. Connect the team to your GitHub organisation (Team settings → Git → Install the Netlify GitHub app on the transferred repository).
- 2 **Preferred:** accept the *site transfer* of `rc-pbt` from Rethink Reality (Path A). **Alternative:** Add new project → Import from Git → pick the repository. Build settings are read from `netlify.toml` automatically (build command `npm ci && npm run build`, publish `dist`, functions `netlify/functions`, Node 20).

- 3 Site configuration → Environment variables: add every variable in Section 4. Mark the secret ones as *secret* so they are hidden in the UI and logs.
- 4 Trigger a deploy (Deploys → Trigger deploy → Clear cache and deploy). Wait for "Published".
- 5 Optional: Domain management → add your custom domain and let Netlify issue the certificate. Then update the Supabase Site URL / redirect URLs (3.1 step 5) and `APP_BASE_URL` (Section 4) to match.

No other Netlify products are used. Forms, Identity, Blobs and scheduled functions are not required. Everything is standard hosting plus Functions.

3.3 Google AI Studio — Gemini API key

What it provides. One API key from Google AI Studio powers every AI feature:

MODEL	USED FOR	CALLED FROM
<code>gemini-3-flash-preview</code>	AI customer replies in text role-play; ACT scorecard evaluation; in-chat coach hints; Pet Vision photo analysis (breed, body-condition, dermatology); Fecal Scan (stool photo scoring)	Netlify Functions
<code>gemini-3.1-flash-live-preview</code>	Live voice role-play (real-time audio in both directions, 5-minute cap per session)	Device, using a single-use token minted by a Netlify Function
<code>gemini-embedding-001</code>	Embeddings for the knowledge base (ingest and retrieval), 768 dimensions	Netlify Functions

Plan guidance. The free tier of the Gemini API has low rate limits and is not suitable for a team of trainees using voice mode. Enable billing on the Google Cloud project behind the key ("paid tier") and set a monthly budget alert. Check current pricing at ai.google.dev/pricing.

Steps

- 1 Sign in to aistudio.google.com with a Google Workspace account owned by your company (not a personal Gmail).
- 2 Get API key → Create API key → create it in a **new, dedicated Google Cloud project** (for example `pbt-production`) so its usage and billing are isolated.
- 3 In the Google Cloud console for that project: Billing → link a billing account; Budgets & alerts → set a monthly budget with email alerts.

- 4 APIs & Services → Credentials → edit the key → API restrictions → restrict it to the *Generative Language API* only. Do **not** add HTTP-referrer restrictions: every call with this key comes from Netlify Functions, which send no referrer, so a referrer restriction would block all AI features.
- 5 Paste the key into Netlify as `GEMINI_API_KEY` (Section 4) and redeploy. Functions read it at runtime, but Netlify only applies changed environment variables to new deploys, so redeploy whenever it changes.

Model IDs are "preview" releases. Google retires preview models on its own schedule. When that happens the AI features return errors until the model IDs in `src/shared/ai/models.ts` (the single file that holds all three) are updated and redeployed. Watch Google's deprecation notices and the Admin → Analytics → AI quality screen (failure-rate alert).

3.4 GitHub — source code

What it provides. The repository holds the full source (consumer app, admin dashboard, Netlify Functions, database migrations, tests and this documentation). Netlify watches the `main` branch and deploys every push. There are no GitHub Actions workflows to migrate; the test suite (about 1,140 tests) runs locally with `npm test`.

Steps

- 1 Create a GitHub organisation for your company (free tier is sufficient) and enable 2FA enforcement.
- 2 Accept the repository transfer of `RethinkRealityAI/PBT` (Path A). GitHub keeps redirects from the old URL, but re-link Netlify to the new location anyway.
- 3 Settings → Branches: protect `main` (require a pull request before merging) so nobody deploys to production by accident.

3.5 Email provider — Resend (default) or SMTP

What it provides. PBT sends its own branded transactional emails rather than using Supabase's mailer: *Admin invitation*, *Role changed*, *Welcome*, *Account disabled* and *Password reset*. The templates are editable in Admin → Email → Templates, and every send is logged in Admin → Email → Delivery. Without a provider configured, admin invitations still work (the accept link is shown on screen for manual copy) but password-reset and welcome emails cannot be delivered.

Choose one: **Resend** is the simplest (HTTPS API, generous free tier, one API key). **SMTP** works with any provider you already have (Microsoft 365, Google Workspace relay, SendGrid, Postmark, Mailgun).

Steps (Resend)

- 1 Create an account at resend.com with your company identity; enable 2FA.

- 2 Domains → Add domain (for example `mail.yourclinic.com`) and add the DNS records it gives you (SPF, DKIM, and the return-path CNAME). Wait for "Verified". Sending from an unverified domain lands in spam or is rejected.
- 3 API keys → Create API key with *Sending access* only.
- 4 In PBT: Admin → Email → Settings → provider *Resend*, paste the key, set From address (on the verified domain), From name, Reply-to and the App base URL → Save → use the test-send. The key is encrypted at rest; alternatively set `RESEND_API_KEY` in Netlify (Section 4).

Steps (SMTP)

- 1 Obtain host, port (587 STARTTLS is the default; 465 for implicit TLS with "secure" on), username and password from your provider, and make sure the From address is authorised to send.
- 2 Enter them in Admin → Email → Settings (provider *SMTP*) or as the `SMTP_*` environment variables in Section 4, then test-send.

3.6 Notion — admin documentation system

What it provides. Nothing in the running platform. Notion hosts the *PBT Admin Documentation* workspace that accompanies this handover: a getting-started guide for admins, one page per admin-dashboard section explaining every screen and action, the roles and permissions reference, operations runbooks, a glossary, and the handover checklist you can tick off. Rethink Reality built it in its own Notion and will transfer it to yours. Current location: `app.notion.com/p/3d0dd85c630c8151bb66dcfb92257fae` (page "PBT Admin Documentation").

Steps

- 1 Create a Notion workspace for your company (Free or Plus is enough) and invite the people who will administer PBT.
- 2 Receive the documentation: either accept the shared page and use Notion's *Move to* → your workspace, or import the exported copy (Settings → Import → HTML/Markdown) that Rethink Reality provides.
- 3 Assign an owner for the documentation. Every time a setting, role or process changes in the admin dashboard, update the matching page. The docs are only useful if they stay current.

Environment variables — complete reference

All variables are set in Netlify → Site configuration → Environment variables. Variables prefixed `VITE_` are compiled into the browser bundle at build time and are public by design; all others, including `GEMINI_API_KEY`, are read only by Netlify Functions at runtime. **Any change to any variable needs a redeploy to take effect.** Never create a `VITE_GEMINI_*` variable: the `VITE_` prefix would publish the key in the bundle.

Required

VARIABLE	SCOPE	PURPOSE	WHAT BREAKS IF MISSING
<code>GEMINI_API_KEY</code>	Functions (secret)	Google AI Studio key for all Gemini calls	Every AI feature: role-play, scoring, coach hints, voice, Pet Vision, Fecal Scan, knowledge ingest/retrieval
<code>VITE_SUPABASE_URL</code>	Build (public)	Supabase project URL	Cloud sync, sign-in and the whole admin dashboard. The consumer app still works anonymously with local storage only.
<code>VITE_SUPABASE_PUBLISHABLE_KEY</code>	Build (public)	Supabase publishable (anon) key	Same as above
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Functions (secret)	Server-side access that bypasses RLS for admin reads/writes, score writes, account deletion, invitations, recovery and knowledge-base seeding	Every admin function returns "Server misconfigured"; account deletion and password reset fail

Strongly recommended

VARIABLE	SCOPE	PURPOSE	NOTES
<code>EMAIL_SECRET_KEY</code>	Functions (secret)	32+ random bytes used to encrypt email credentials stored via the admin UI (AES-256-GCM)	If unset, the service-role key is used as the encryption key instead. That works, but rotating the service-role key would then make stored email credentials unreadable until re-entered. Generate with <code>openssl rand -base64 48</code> .

<code>APP_BASE_URL</code>	Functions	Public base URL used inside emails (invite links, reset links)	Falls back to Netlify's own <code>URL</code> . Set it explicitly once you use a custom domain, or configure it in Admin → Email → Settings.
<code>SUPABASE_URL</code>	Functions	Alias of the project URL for server code and the <code>verify:db</code> script	Functions fall back to <code>VITE_SUPABASE_URL</code> ; set both to keep tooling simple.

Email (only if you prefer env vars over the admin Settings screen)

VARIABLE	PURPOSE
<code>RESEND_API_KEY</code>	Resend sending key. If present and no SMTP host is set, Resend is the provider.
<code>SMTP_HOST</code> , <code>SMTP_PORT</code> (default 587), <code>SMTP_USER</code> , <code>SMTP_PASSWORD</code> , <code>SMTP_SECURE</code> (<code>true</code> for port 465)	SMTP relay settings. If <code>SMTP_HOST</code> is set and no Resend key, SMTP is the provider.
<code>EMAIL_FROM_ADDRESS</code> , <code>EMAIL_FROM_NAME</code> , <code>EMAIL_REPLY_TO</code>	Sender identity shown to recipients. Reply-to doubles as the support address in templates.

Values saved in Admin → Email → Settings take precedence over environment variables; the screen shows which source is active.

Developer / local only

VARIABLE	PURPOSE
<code>VITE_ADMIN MOCK</code>	Runs the admin dashboard against mock data for UI development. Never set in production.
<code>PBT_ALLOW_DEV_SYNC</code>	Set to <code>1</code> to let <code>netlify dev</code> run the knowledge-base sync locally. Never set in production.

Handover sequence

There are two ways to take ownership. **Path A (transfer)** moves the existing, working resources into your accounts and keeps all data, history and configuration. It is faster, lower risk and is what we recommend. **Path B (rebuild)** creates everything fresh from the source code and migrations; use it only if your policy forbids accepting transferred resources or you want a clean slate with no historical data.

Path A — Transfer (recommended)

Supabase project transfer, Netlify site transfer, GitHub repository transfer. Zero data loss, no downtime, roughly half a day including verification.

Path B — Rebuild

New Supabase project + run 23 migrations, new Netlify site from Git, fresh env vars. Existing users and session history are not carried over unless exported and re-imported.

Phase 0 — Prepare (client, before handover day)

- ☐ Create the accounts in Section 3 using company identities with 2FA.
- ☐ Create the Gemini API key with billing enabled (3.3) and have it ready.
- ☐ Set up and verify your sending domain in Resend or gather SMTP credentials (3.5).
- ☐ Decide the production domain (keep `*.netlify.app` or use a custom domain) and who your first *owner* admins will be (at least two people).
- ☐ Send Rethink Reality the email addresses of your Supabase organisation owner, Netlify team owner and GitHub organisation owner so transfers can be initiated.

Phase 1 — Move the resources

Path A

- 1 GitHub:** Rethink Reality transfers `RethinkRealityAI/PBT` to your organisation; you accept the transfer email.
- 2 Supabase:** Rethink Reality initiates Project Settings → General → Transfer project to your organisation; your organisation owner accepts. The project URL, keys and data remain unchanged, so no redeploy is needed for this step.
- 3 Netlify:** Rethink Reality transfers site `rc-pbt` to your team (Site configuration → General → Transfer site). Environment variables move with the site. Re-link the repository to its new GitHub location (Site configuration → Build & deploy → Continuous deployment).

Path B

- 1** Fork or copy the repository into your organisation (Rethink Reality can push it directly).

- 2 Create the Supabase project; in the SQL editor run each file in `supabase/migrations/` in filename (timestamp) order, from `20260504000000_init.sql` through `20260923000000_knowledge_sync_lease.sql`. Every migration is idempotent, so re-running is safe. The knowledge base then seeds itself after the first production deploy; there is nothing to load by hand.
- 3 Create the Netlify site from the repository, add the environment variables, deploy.
- 4 Optional data carry-over: Rethink Reality exports `profiles`, `training_sessions`, `user_scenarios`, `knowledge_documents` / `knowledge_chunks`, `flags`, `simulation_config`, `admin_roles` and `email_templates` from the old project and imports them into the new one. Auth users cannot be moved with their passwords; trainees would need to reset passwords or re-register (their local progress on-device is unaffected).

Phase 2 — Re-key and configure

- 1 In Netlify, replace `GEMINI_API_KEY` with your key. Add `EMAIL_SECRET_KEY` and `APP_BASE_URL`. Confirm the four required variables are present.
- 2 In Supabase, rotate the service-role key (Project Settings → API → Generate new key) so Rethink Reality no longer holds a valid one, then update `SUPABASE_SERVICE_ROLE_KEY` in Netlify. Do this *before* entering email credentials so nothing has to be re-encrypted.
- 3 Set the Supabase Auth Site URL and redirect URLs to your production domain (3.1 step 5).
- 4 Deploys → Trigger deploy → *Clear cache and deploy site*. Wait for "Published".
- 5 **Bootstrap your first owner.** Open the app, tap *Save your progress* (or Settings → Create account) and sign up with the owner's email. Then in the Supabase SQL editor run the statement below, replacing the email. A database trigger promotes the row to the `owner` role. Repeat for your second owner, then sign in at `/admin`.

```
update public.profiles
  set is_admin = true, admin_role = 'owner'
 where user_id = (select id from auth.users where email = 'owner@yourclinic.com');
```

- 6 In Admin → Email → Settings, enter the provider credentials, sender identity and App base URL, save, and send a test email to yourself.
- 7 In Admin → People → Invites, invite the rest of your admin team with appropriate roles (see the Notion "Roles & permissions" page). Owners should be few; most staff need *Content manager*, *Analyst* or *Support*.

Verification before go-live

Run these checks on the new deployment. Every item should pass before you announce the platform to trainees. Anything that fails points at a specific section above.

Automated

CHECK	COMMAND (FROM A CLONE OF THE REPO)	PROVES
Schema applied	<code>SUPABASE_URL=... SUPABASE_SERVICE_ROLE_KEY=... npm run verify:db</code>	Every table the code references exists in the live database (all 23 migrations applied)
Test suite	<code>npm ci && npm test</code>	About 1,140 tests pass, including migration/code parity, translation catalog guards and the Fecal Scan grounding rules
Bundle budget	<code>npm run build && npm run check:bundle</code>	Main entry stays under the 500 kB gzip budget (currently about 60 kB) and that no API-key-shaped string is in the bundle

Manual smoke test (15 minutes)

- ☐ **Consumer:** open the site in a private window → accept terms → complete the ECHO quiz → land on Home with a driver colour.
- ☐ **Text role-play:** start Today's pick → exchange three messages → end session → scorecard shows real scores (not "score unavailable").
- ☐ **Voice role-play:** tap Begin in voice mode → browser asks for the microphone → the AI customer speaks → end → transcript and score appear.
- ☐ **Pet Analyzer:** upload a pet photo → breed and body-condition estimate returned.
- ☐ **Fecal Scan:** Home → Fecal Scan → pick *Adult dog* → photograph a stool (or a printed chart photo) → a score from 1 to 5 appears with a confidence rating, the chart's reference photo, and the chart passages it was grounded in.
- ☐ **Account:** Save your progress → sign up → refresh → progress persists; Settings → sign out → sign in.

- ☐ **Password reset:** Sign in → Forgot password → email arrives from your domain → link opens the reset screen.
- ☐ **French:** Settings → switch to Français → Home and a role-play render in French.
- ☐ **Admin:** sign in at `/admin` as owner → Overview shows counts → People shows your users → Analytics → AI quality shows the sessions you just ran with cost and latency.
- ☐ **Admin invite:** invite a colleague → email arrives → accept link creates their admin access with the chosen role.
- ☐ **Flags:** toggle a component flag (for example the Home library card) → the consumer app reflects it on reload, with no deploy.
- ☐ **Knowledge:** a few minutes after the first production deploy, Admin → Library → Knowledge lists the built-in documents with chunk counts (they seed themselves; there is no button). Use *Try a search* to confirm retrieval works, which proves the embedding model and the vector extension.
- ☐ **Email log:** Email → Delivery shows the test, reset and invite messages as sent.

Security & ownership checklist

- ☐ Two-factor authentication enforced on Supabase, Netlify, GitHub, Google and the email provider.
- ☐ At least two *owner* admins in PBT and at least two owners on every hosted account. The platform refuses to demote the last active owner, but hosting accounts do not protect you from a single lost login.
- ☐ Rethink Reality members removed from the Supabase organisation, Netlify team, GitHub organisation and the PBT admin team at close-out (Section 10).
- ☐ Supabase service-role key rotated after transfer (Phase 2 step 2). Gemini key replaced with yours. Old Rethink Reality Gemini key deleted on our side. That old key shipped in the browser bundle before the September security release, so treat it as exposed and make sure it is deleted, not just replaced.
- ☐ `EMAIL_SECRET_KEY` set to a dedicated random value and stored in your password manager. If it is ever lost, re-enter the email credentials in the admin screen; nothing else is affected.
- ☐ Gemini key restricted to the Generative Language API with a budget alert. The key lives only in Netlify Functions; voice mode connects with a single-use token per session, AI endpoints are rate-limited per IP, and scores are written by the server (a database trigger rejects scores written from a browser). Still monitor usage in Google Cloud and rotate if you see unexplained spend.
- ☐ Old Netlify deploy previews reviewed. Every deploy stays reachable at its own link with the environment it was built with. Delete the Deploy Preview for commit `eccee53` (pull request #23) in Netlify → Deploys; rotating the service-role key in Phase 2 also neutralises it.
- ☐ Supabase → Authentication → Rate limits reviewed; Database → Backups → daily backups confirmed (Pro plan) and Point-in-time recovery considered.
- ☐ Branch protection on `main`; only reviewed pull requests reach production.
- ☐ Privacy: trainees can opt out of AI-training use in Settings, and can delete their own account (self-service, wipes all their data). Make sure your privacy notice reflects that data is stored in Supabase (US East) and sent to Google for AI processing.

Ongoing operations, costs and monitoring

AREA	WHERE TO LOOK	WHAT TO WATCH	CADENCE
AI health & spend	Admin → Analytics → AI quality; Google Cloud → Billing	Failure-rate alert banner, latency, cost per session, per-model breakdown; monthly Google invoice vs. budget	Weekly
Database	Supabase → Reports / Usage	Database size, egress, auth users, backups succeeding	Monthly
Hosting	Netlify → Usage & billing; Deploys; Functions logs	Bandwidth, function invocations, failed deploys, function errors ("Server misconfigured" means an env var is missing)	Monthly and after every deploy
Email deliverability	Admin → Email → Delivery; Resend dashboard	Failed sends, bounces, domain still verified	Monthly
Model deprecations	Google AI release notes	Retirement dates for the three model IDs in Section 3.3	Quarterly
Dependencies	GitHub → Dependabot alerts	Security advisories for npm packages	Monthly
Content	Admin → Library and Simulation	Scenario rotation, knowledge base freshness, scoring weights still reflect your training goals	Quarterly

Cost drivers

- **Gemini usage is the only usage-based cost that scales with trainees.** Voice sessions cost the most per minute (hence the 5-minute cap); text sessions, scoring and Fecal Scans (two image calls each) are cheap; embeddings are a one-off per knowledge document.
- Supabase and Netlify are flat monthly plans with usage overage only at high volume.
- Resend's free tier covers a small admin team; SMTP through an existing provider is usually already paid for.

Making changes safely

- **No-deploy changes** (admin dashboard): scenarios, knowledge base, simulation tuning (prompts, personas, scoring weights), feature flags, email templates, roles and users.
- **Code changes** (developer): open a pull request → tests pass → merge to `main` → Netlify deploys automatically. If a change adds a database table or column, apply its migration in Supabase *before* merging and run `npm run verify:db`.

- **Translations:** every user-facing string exists in English and Canadian French; a missing French key fails the build on purpose.

Known outstanding work

The platform is production-complete for its scope. These items are documented in the repository as polish or future hardening and are handed over as-is:

ITEM	STATUS	IMPACT
Accessibility sweep	Labels, live regions and keyboard basics done; modal focus traps and a Lighthouse ≥ 90 target remain	Low; affects assistive-technology users
French Terms & Conditions legal review	Translated clause-for-clause, not yet reviewed by counsel	Legal; review before relying on the French terms
French live-voice smoke test	Untested against the real Gemini Live socket; fr-FR fallback in place	Low; English voice fully tested
Email verification at sign-up	Built, switched off by product decision (<code>FLAGS.EMAIL_VERIFICATION</code>)	None unless you want verified emails; enabling also needs Supabase "Confirm email" on
Inclusive-writing pass (French quiz)	A few masculine-default adjectives flagged in the catalog	Cosmetic
Fecal Scan clinical agreement check	Checked against the chart photos and test cases, not yet against a veterinarian's blind scoring of real clinic photos	Recommended before presenting it as more than a supportive aid
Fecal Scan off switch	The admin flag hides the desktop sidebar entry only; the Home tile and Pet Analyzer link stay visible	Low; a feature-wide flag can be added if you need to switch it off entirely

Handover sign-off

Handover is complete when every row below is initialled by both parties. Until then Rethink Reality retains access solely to assist with the transfer.

MILESTONE	CLIENT	RETHINK REALITY	DATE
Accounts created with 2FA (Section 3)			
GitHub repository transferred and re-linked			
Supabase project transferred (or rebuilt and migrated)			
Netlify site transferred, env vars complete, deploy published			
Gemini key replaced; service-role key rotated			
Two client owners active in Admin; email provider tested			
Verification checklist passed (Section 6)			
Notion documentation received and owned			
Rethink Reality access removed everywhere			

Support after handover

Questions about this document or the platform: **dapo@rethinkreality.ai**. Reference material handed over with the platform: this PDF, the Notion documentation workspace, the repository's `README.md`, `CLAUDE.md` (engineering guide) and the design specification under `docs/`.